Bases de Data Engineering et Machine Learning (1)

Séance 3 : Validation automatisée des données

BAC+4 Data | Mouaad AGOURRAM | 5 novembre 2026

Où en est-on



Ce que l'ingestion seule ne garantit pas

Les colonnes sont-elles celles qui étaient attendues ?
Les types sont-ils les bons : nombre, texte, date ?
Les valeurs respectent-elles les règles du métier ?

Au programme

01 Pourquoi valider les données

02 Types et inférence

03 PyArrow, Parquet et schéma explicite

04 Contrôle de schéma

05 Contraintes et tests de qualité

06 Démo et mini-projet Boutique

Pourquoi valider les données

Un pipeline vert, des données fausses

CE QUE LE SUCCÈS GARANTIT

- Le code s'est exécuté jusqu'au bout
- Les fichiers ont été lus et écrits
- Aucune exception n'a été levée

CE QU'IL NE GARANTIT PAS

- Les colonnes attendues sont présentes
- Les types sont les bons
- Les valeurs ont du sens

Les dimensions de la qualité

Dimension	La question	Exemple
Complétude	Les valeurs attendues sont-elles là ?	E-mail du client vide
Validité	Le format et le domaine sont-ils respectés ?	Mois entre 1 et 12
Unicité	Chaque entité apparaît-elle une seule fois ?	Identifiant de commande en double
Cohérence	Les données s'accordent-elles entre elles ?	Une commande sans client connu
Fraîcheur	Les données sont-elles assez récentes ?	L'export du jour est présent

Où placer les contrôles



Bloquer

Schéma non conforme, clé en double : le pipeline s'arrête et personne ne consomme la donnée.

Signaler

Quelques valeurs manquantes sous le seuil toléré : on continue et on garde la trace.



Types et inférence

Les pièges de l'inférence

Colonne	On attend	L'outil peut lire
Code postal : 01000	Texte	Entier 1000 : le zéro de tête disparaît
Montant : 12,50	Décimal	Texte : la virgule empêche la conversion
Âge, avec des valeurs vides	Entier	Décimal : le vide impose ce type

Le piège, c'est le silence

Aucune erreur n'est levée : le mauvais type se découvre au moment du calcul.

L'inférence est une hypothèse

- L'outil lit un échantillon et en déduit un type
- Deux outils peuvent déduire deux types différents
- Une seule valeur atypique change le type de toute la colonne
- Un code d'absence (-999, N/A) est lu comme une valeur ordinaire

Typer, c'est décider
Le type doit être écrit par nous, pas deviné par l'outil.

Convertir sans masquer les échecs

Convertir

1

Transformer chaque valeur vers le type attendu

Compter les échecs

2

Mesurer ce qui n'a pas pu être converti

Décider

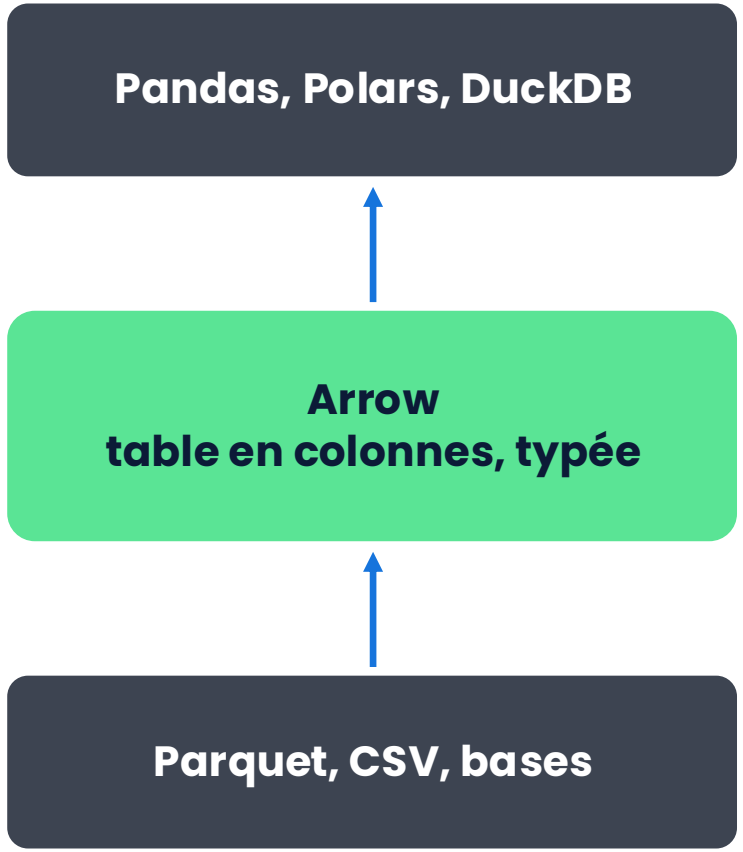
3

Comparer le compte à un seuil : continuer ou arrêter

PyArrow, Parquet et schéma explicite

Arrow et PyArrow

- Arrow : un tableau représenté en mémoire, colonne par colonne
- PyArrow : la bibliothèque Python de ce format
- Chaque colonne a un type déclaré et une nullabilité
- Pandas, Polars et DuckDB lisent et produisent des tables Arrow



Parquet, un format avec schéma

CSV

- Texte brut, sans types déclarés
- Lu ligne par ligne
- Aucun schéma dans le fichier

PARQUET

- Binaire en colonnes, compressé
- Le schéma est écrit dans le fichier
- On peut ne lire que les colonnes utiles

Le schéma d'une table

Élément	Rôle
Nom	Identifie la colonne
Type	Dit ce que contient la colonne
Nullabilité	Dit si une valeur vide est permise

Type	Usage
Entiers (int8 à int64)	Codes, quantités
Décimal flottant	Mesures
Texte (string)	Identifiants, libellés
Décimal exact	Montants
Date, heure (timestamp)	Instants, avec fuseau

Imposer le schéma à la lecture

Déclarer

1

Écrire les types attendus, le séparateur, la virgule décimale et les codes d'absence

Lire

2

Le lecteur convertit chaque valeur vers le type déclaré

Réussir ou échouer

3

Conversion réussie, ou erreur claire avec la valeur en cause

Parquet : écrire et relire

Écrire

Le schéma est enregistré dans le fichier avec les données

Lire le schéma seul

Sans charger les données, très rapide

Lire les colonnes utiles

Seules les colonnes demandées sont lues

Relire

Aucun type n'est deviné : le fichier porte son contrat

Pandas et Arrow

Aspect	Pandas par défaut	Pandas avec backend Arrow
Types	Types propres à Pandas	Types Arrow, comme le schéma
Valeurs manquantes	Un entier devient décimal	Valeurs nulles, entiers conservés
Échange avec PyArrow	Conversion à prévoir	Plus direct

Le schéma ne change pas d'un outil à l'autre

On choisit l'outil selon l'étape ; le contrat de types reste le même.



Contrôle de schéma

Ce qu'on contrôle dans un schéma

Colonnes présentes

Toutes les colonnes attendues sont là

Colonnes en trop

Une colonne inattendue est signalée

Types

Chaque colonne a le type attendu

Nullabilité

Une colonne obligatoire est toujours renseignée

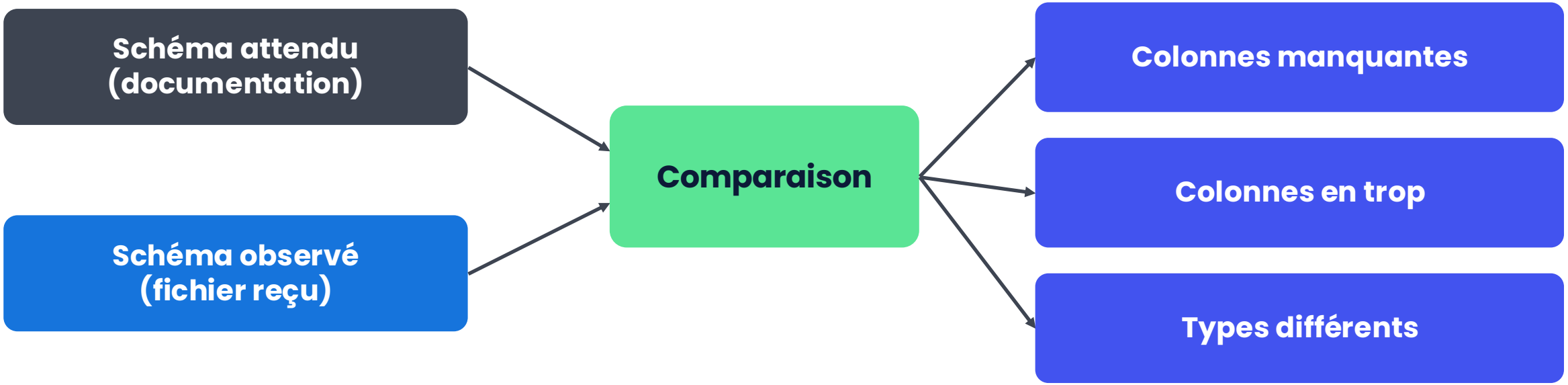
La dérive de schéma

Changement	Exemple	Effet sur le pipeline
Colonne renommée	prix devient montant	Erreur, ou lecture de vide
Colonne supprimée	date_naissance disparaît	Calculs en aval faux
Type modifié	Un nombre devient du texte	Conversion en échec
Colonne ajoutée	Nouvelle colonne en fin de fichier	Souvent sans effet, à signaler

La source évolue sans nous

Le contrôle de schéma est la première barrière, avant toute lecture de valeurs.

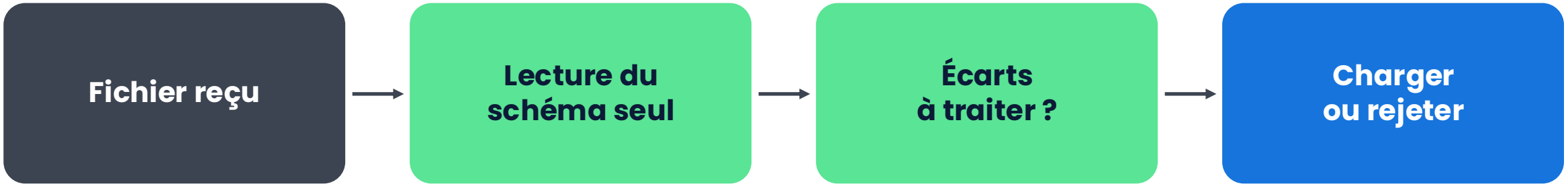
Comparer attendu et observé



Un résultat détaillé

Un simple oui ou non ne dit pas quoi réparer : on rend ce qui a changé.

Vérifier avant de charger



Rejeter tôt

Un fichier refusé à l'entrée coûte une alerte. Un fichier accepté à tort peut coûter un tableau de bord faux pendant des semaines.

Contraintes et règles métier

Les familles de contraintes

Famille	La règle	Exemple
Non-nullité	La colonne est obligatoire	Identifiant jamais vide
Unicité	Pas de doublon sur la clé	Identifiant de commande unique
Plage	La valeur reste entre deux bornes	Mois entre 1 et 12
Domaine	La valeur appartient à une liste	Statut parmi quatre valeurs
Format	La valeur suit un motif	Code postal à 5 chiffres

Portée d'une règle

Portée	La question	Exemple
Une colonne	La valeur est-elle valide seule ?	Âge entre 0 et 120
Entre colonnes	Les valeurs s'accordent-elles ?	Date de fin après la date de début
Entre tables	Les références existent-elles ?	Chaque commande a un client connu

Une règle, un compte
Chaque règle pose une question aux données et rend un nombre de lignes en défaut.

Bloquer ou signaler

ERREUR BLOQUANTE

- Clé en double
- Colonne obligatoire absente
- Type non convertible

AVERTISSEMENT

- Valeurs manquantes sous un seuil
- Valeur hors domaine rare
- Colonne en trop



Tests de qualité automatisés

Du contrôle ponctuel au test

CONTRÔLE PONCTUEL

- Affiche un résultat dans la console
- Dépend de quelqu'un qui regarde
- Disparaît après l'exécution

TEST AUTOMATISÉ

- Renvoie un résultat structuré
- S'exécute à chaque lancement
- Peut arrêter le pipeline

Les outils

assert

Arrête le script si une condition est fausse. Simple, pour un contrôle isolé.

pytest

Lance tous les tests et rapporte les échecs. Adapté au code et aux données.

Pandera

Décrit le schéma et les règles d'un DataFrame. Un aperçu en démonstration.

Le compte rendu de qualité

Règle	Verdict	Lignes en défaut	Gravité
id_releve unique	ÉCHEC	3	Bloquante
humidité entre 0 et 100	TOLÉRÉ	8	Avertissement
capteur connu	ÉCHEC	2	Bloquante

Extrait du compte rendu de la démonstration

Trois verdicts

OK : aucune ligne en défaut. TOLÉRÉ : des lignes en défaut, sous le seuil accepté. ÉCHEC : au-dessus du seuil.
Une règle bloquante en échec arrête le pipeline.

Démo

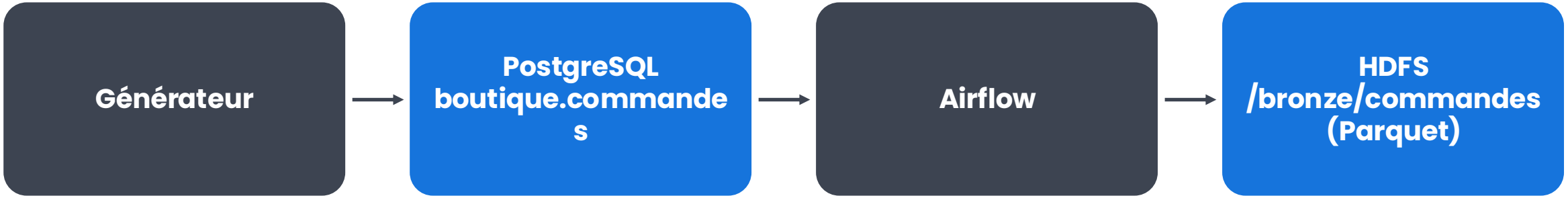
Valider une livraison de relevés

seance3/demo_validation.ipynb

Durée : 15 min

Mini-projet Boutique

La mission Boutique



La question du jour

Peut-on faire confiance à ces données avant de les déposer en zone bronze ?

Étape 1 : le schéma attendu

Colonne	Type SQL	Type Arrow
id	BIGINT	int64
client_id	INTEGER	int32
produit	TEXT	string
quantite	INTEGER	int32
montant	NUMERIC(10, 2)	decimal128(10, 2)
statut	TEXT	string
updated_at	TIMESTAMPTZ	timestamp avec fuseau

> **La règle**
> Le schéma vient de la documentation de la source, pas de ce que l'outil devine.

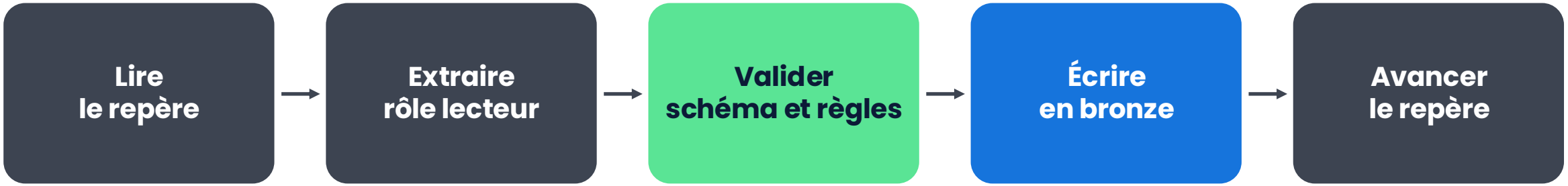
Étape 2 : les règles de qualité

Règle	Si elle est violée	Gravité
id unique	Comptages de commandes faux	Bloquante
quantite positive	Totaux faussés	Bloquante
montant positif ou nul	Chiffre d'affaires faussé	Bloquante
statut parmi quatre valeurs	Répartition des statuts cassée	Bloquante
aucune valeur vide	Totaux et incrémental faussés	Bloquante

Déjà dans la base, à vérifier quand même

La source peut changer, et vos propres sources n'auront pas ces garde-fous.

Étape 3 : brancher les contrôles



Où écrire le code

Dans le DAG ingestion_commandes.py, vide dans le dossier fourni : on l'écrit ensemble.
Les journaux peuvent s'appuyer sur scripts/socle.py ; un compte rendu est enregistré à chaque exécution.

Votre projet en quatre questions

1. Ma source

Où sont mes données, et quelle documentation les décrit ?

2. Mon schéma

Quel type attendu pour chaque colonne ?

3. Mes règles

Quelles règles découlent de mon problème ?

4. Mes contrôles

Où les placer, et que faire en cas d'échec ?

Ce qu'il faut retenir

- Un mauvais type ne fait pas échouer un pipeline : il faut le vérifier
- Un schéma explicite remplace l'inférence
- Parquet conserve le schéma avec les données
- On contrôle le schéma avant les valeurs, et on rejette tôt
- Une règle bloque ou signale, et laisse une trace

Pour la suite

Travail autonome

Créer des contrôles automatiques et un compte rendu de qualité
sur le jeu de données de votre projet.

Séance 4 : 12 novembre 2026

Pipelines Python reproductibles :
ingestion, validation, transformation, journaux et gestion des erreurs.

Merci, des questions ?

